ЧАСТЬ 3 ИЗ 3

RUST

От владения к реальным программам

Доводим модель памяти до конца — и собираем из конструкций настоящие программы

Владение и функции	**Ссылки в структурах**	**Lifetimes**	**Память**
Option / Result	Traits	Модули и Cargo	ООП и ФП
Ввод-вывод	Потоки	Макросы	

Владение и функции: отдать, вернуть, одолжить

```
fn consume(s: String) { /* ... */ }  // забирает
fn give_back(s: String) -> String {  // берёт и
    s                                //   возвращает
}
fn borrow(s: &str) -> usize {        // одалживает
    s.len()
}

let s1 = String::from("hello");
let n = borrow(&s1);      // s1 — всё ещё владелец
let s2 = give_back(s1);   // s1 → функция → s2
consume(s2);              // s2 больше нет
```

`s: String`

Забирает: значение уничтожится внутри функции

`s: String → String`

Берёт и возвращает: работает, но неудобно

`s: &str`

Одалживает: владелец остаётся снаружи

Читает — ссылка. Хранит у себя — владение. Отдаёт наружу — возвращаемое значение.

Ссылки в структурах

Структура владеет

```
struct User {
    name: String,           // владеет
}

let name = String::from("Alice");
let u = User { name };      // name переехала
```

```
let r;
{
    let tmp = String::from("Tmp");
    r = UserRef { name: &tmp };
}                           // tmp уничтожена
println!("{}", r.name);     // ← ошибка
```

Структура ссылается: параметр 'a

```
struct UserRef<'a> {
    name: &'a str,          // ссылается
}

let name = String::from("Bob");
let r = UserRef { name: &name };
println!("{} {}", name, r.name);  // ок
```

```
error[E0597]: `tmp` does not live long enough
```

'a ничего не продлевает: он только описывает связь «структура не переживёт данные», а компилятор её проверяет.

код: part3/03_struct_refs.rs

Ссылки на элементы коллекций

```
let mut names = vec![String::from("Alice"),
                     String::from("Bob")];

for n in &names { println!("{n}"); } // ссылки

let first = &names[0];        // &String
println!("{first}");          // последнее использование
names.push(String::from("Carol")); // ок

for n in names { println!("{n}"); } // потребляет
```

А так — нельзя

```
let first = &names[0];
names.push(String::from("Dan"));
println!("{first}"); // ← ошибка
```

E0502: cannot borrow as mutable

push может переселить буфер вектора — и first указывала бы в освобождённую память. Запрет действует, пока ссылка используется.

Vec<String> владеет строками: &names — перебор ссылок, names — перебор с передачей владения.

Lifetimes

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn first_word(s: &str) -> &str {  // 'a выводится сам
    s.split(' ').next().unwrap_or("")
}
```

```
let r;
let a = String::from("long string");
{
    let b = String::from("xyz");
    r = longest(&a, &b);
}                     // b уничтожена
println!("{r}");      // ← ошибка
```

'a — контракт

Результат действителен, пока действительны оба аргумента

Не продлевает жизнь

Аннотация описывает связь, данные она не меняет

Выводится сам

В простых случаях аннотации не нужны (first_word)

```
E0597: `b` does not live long enough
```

ownership — кто отвечает · borrowing — кто пользуется · lifetimes — как долго

код: part3/05_lifetimes.rs

Управление памятью: общая картина

Стек

Размер известен заранее: числа, bool, структуры из них. Исчезает при выходе из функции.

Куча

Размер меняется: текст String, элементы Vec. Освобождается через drop владельца.

`Box<T>`

Значение в куче, один владелец

`Rc<T>`

Счётчик ссылок: несколько владельцев в одном потоке

`Arc<T>`

То же для нескольких потоков (слайд 13)

`unsafe`

Сырые указатели, вызов C: проверку берёт на себя программист

```
let b = Box::new(5);                        // в куче
let shared = Rc::new(String::from("общая"));
let other = Rc::clone(&shared);             // владельцев: 2
```

Счётчик Rc дошёл до нуля → drop → память освобождена. В обычном прикладном коде unsafe не нужен.

код: part3/06_memory.rs

Option и Result: match и if let

```
let names = ["Анна", "Олег"];
let pos = names.iter().position(|n| *n == "Олег");
match pos {                          // Option<usize>
    Some(i) => println!("индекс {i}"),
    None => println!("не найден"),
}

let age = "21".parse::<u8>();    // Result<u8, _>
if let Ok(a) = age {
    println!("возраст {a}");
}
```

`Option<T>`

Some(v) или None — вместо null

`Result<T, E>`

Ok(v) или Err(e) — вместо исключений

`match`

Исчерпывающий: None и Err не забыть

`if let`

Когда интересен один вариант

Option и Result — обычные enum из части 2. Взять число из Option<usize> «просто так» нельзя.

Оператор ?: проброс ошибки

```rust
use std::num::ParseIntError;

fn sum(a: &str, b: &str) -> Result<i32, ParseIntError> {
    let x: i32 = a.trim().parse()?;  // Err → return Err
    let y: i32 = b.trim().parse()?;
    Ok(x + y)
}

sum("2", "3");     // Ok(5)
sum("2", "abc");   // Err(ParseIntError { kind: InvalidDigit })
```

Функция с ? обязана возвращать Result (или Option) с совместимым типом ошибки.

`Ok(v)` → `v`

Достаёт значение и идёт дальше

`Err(e)` → `return`

Сразу возвращает ошибку вызывающему

`Option` тоже

None → функция возвращает None

unwrap() / expect() на Err — паника. Result сам по себе обработку не гарантирует.

код: part3/08_question_mark.rs

Traits: общее поведение

Контракт и реализация

```
trait Summary {
    fn summary(&self) -> String;
}

struct Article { title: String }
struct Tweet { text: String }

impl Summary for Article {
    fn summary(&self) -> String {
        format!("Статья: {}", self.title)
    }
}
// impl Summary for Tweet { ... }
```

Полиморфизм

```
// статическая диспетчеризация
fn print_one<T: Summary>(item: &T) {
    println!("{}", item.summary());
}

// динамическая диспетчеризация
let feed: Vec<Box<dyn Summary>> = vec![
    Box::new(article),
    Box::new(tweet),
];
for i in &feed { println!("{}", i.summary()); }
```

trait ≈ interface / protocol: передаёт поведение, а не поля.

Модули, crates, packages и Cargo

```
mod geometry {                    // пространство имён
    pub struct Point { pub x: f64, pub y: f64 }

    pub fn dist(a: &Point, b: &Point) -> f64 {
        helper(a.x - b.x, a.y - b.y)
    }
    fn helper(dx: f64, dy: f64) -> f64 { // приватная
        (dx * dx + dy * dy).sqrt()
    }
}

use geometry::{dist, Point};    // импорт
```

```
[package]
name = "app"
version = "0.1.0"

[dependencies]
rand = "0.8"
```

```
cargo build
cargo run
cargo test
```

Пространство имён → mod

mod geometry { … } или файл geometry.rs

Импорт → use

use geometry::dist; без полного пути

Экспорт → pub

По умолчанию всё приватно; pub use — реэкспорт

Пакет → package / crate

Cargo.toml; crate — корень main.rs (бинарный) или lib.rs (библиотека)

Демо: part3/cargo_demo — lib.rs + main.rs + mecm

код: part3/10_modules.rs

ООП, ФП и протоколы в Rust

```
mod stats {
    pub struct Counter { value: u32 }   // поле приватное

    impl Counter {
        pub fn new() -> Self { Counter { value: 0 } }
        pub fn inc(&mut self) { self.value += 1; }
        pub fn get(&self) -> u32 { self.value }
    }
}

struct Clicker { counter: stats::Counter } // композиция

// c.counter.value = 10;  // E0616: поле приватное
let evens = (0..5).filter(|n| n % 2 == 0).count(); // ФП
```

ООП

struct + impl, инкапсуляция через pub, композиция вместо наследования классов

ФП

Замыкания, итераторы, неизменяемость по умолчанию (часть 2)

Протоколы

trait — контракт, который реализуют разные типы (слайд 9)

Наследования классов нет — это другой набор механизмов, а не недостаток.

код: part3/11_oop.rs

Ввод-вывод: консоль и файлы

Консоль

```
use std::io::{self, Write};

print!("Имя: ");
io::stdout().flush()?;          // показать сразу
let mut name = String::new();
io::stdin().read_line(&mut name)?; // дописывает
println!("Привет, {}!", name.trim());
```

Файлы

```
use std::fs::File;
use std::io::{BufRead, BufReader, Write};

let mut f = File::create("notes.txt")?;
writeln!(f, "первая строка")?;

let file = File::open("notes.txt")?;
for line in BufReader::new(file).lines() {
    println!("> {}", line?);
}
```

```
fn main() -> io::Result<()> {  /* ... */  Ok(()) }
```

Любая операция ввода-вывода может не получиться — поэтому она возвращает Result, а ? пробрасывает ошибку.

Консоль и файл реализуют один трейт Write — поэтому writeln! работает с обоими.

Многопоточность: потоки, Arc<Mutex>, каналы

```
let counter = Arc::new(Mutex::new(0));
let mut handles = vec![];
for _ in 0..4 {
    let c = Arc::clone(&counter);   // ещё владелец
    handles.push(thread::spawn(move || {
        *c.lock().unwrap() += 1;    // по одному потоку
    }));
}
for h in handles { h.join().unwrap(); }
// *counter.lock().unwrap() == 4

let (tx, rx) = mpsc::channel();
thread::spawn(move || tx.send("готово").unwrap());
rx.recv().unwrap();                 // "готово"
```

spawn + move

Замыкание — код потока; move отдаёт ему захваченное

join() → Result

Err — поток завершился паникой

Send / Sync

Компилятор проверяет, что можно передать в поток: Rc — нельзя (E0277)

Гонок данных нет. Дедлоки и логические ошибки — возможны.

код: part3/13_concurrency.rs

Макросы: код, который пишет код

Декларативные: macro_rules!

```
macro_rules! square {
    ($x:expr) => { $x * $x };
}

macro_rules! my_vec {
    ($($x:expr),*) => {{
        let mut v = Vec::new();
        $( v.push($x); )*   // для каждого
        v
    }};
}

square!(1 + 1);      // 4, а не 3
my_vec![1, 2, 3];    // [1, 2, 3]
```

Процедурные: derive

```
#[derive(Debug, Clone, PartialEq)]
struct Point { x: i32, y: i32 }
// Debug, Clone и == сгенерированы
```

println! — тоже макрос

```
println!("{} {}", 1);
// ошибка компиляции:
// 2 места, 1 аргумент
```

Раскрытие — при компиляции. Сгенерированный код выполняется как обычный.

ИТОГ КУРСА

Что делает Rust особенным

Ownership

Кто отвечает за ресурс и когда он освобождается

Borrowing

Доступ через ссылки: много &T или один &mut T

Lifetimes

Ссылка не переживёт данные

Option / Result

Отсутствие значения и ошибка — часть типа

Pattern matching

match разбирает все варианты

Traits

Контракты и полиморфизм без наследования

Макросы

Код генерируется при компиляции

Send / Sync

Многопоточность без гонок данных

Механизмы работают вместе и проверяются компилятором: безопасность памяти без сборщика мусора и скорость системного языка. Тесты и обработка ошибок всё равно нужны.